Dataset A: use the nilearn Oasis dataset for **classification** of male and female subjects based on structural brain scans

```
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker
from nilearn.image import index_img
import nibabel as nib

brain_data = datasets.fetch_oasis_vbm(n_subjects=100)
yeo = datasets.fetch_atlas_schaefer_2018(n_rois=200)  # this needs to be modified for some tasks
print('%i regions found in this atlas' % (len(np.unique(nib.load(yeo.maps).get_data().ravel())) - 1))

masker = NiftiLabelsMasker(labels_img=yeo.maps, standardize=True,
    memory='nilearn_cache')
input_variables = masker.fit_transform(brain_data.gray_matter_maps)
output_variable = np.array(brain_data.ext_vars.mf == 'F', dtype=int)  # gives 1 for females and 0 for males
```

Note for Tasks 1/2/5 and 6/7/10: PolynomialFeatures will generate many features at higher degrees, so use only the first 50 columns of the input data (input_variables[:, 0:50]). If the higher degree polynomial features still cause problems, you can restrict your plot to the lower degrees (eg 1-4).

Task 1- Use sklearn.preprocessing.PolynomialFeatures() to fit LogisticRegression(default parameters) based on 100 brain regions of interest (n_rois=100). Use the first 90 subjects as the training set. Fit 5 different models with increasingly representational capacity (degree=1-5). Next, evaluate each fitted classification model on the unseen 10 subjects, and plot the classification accuracy (x axis=5 different models, y axis=prediction accuracy; plot train/test as lines on the same plot using different colours).

Task 2- Same as task 1, but with Bayesian linear regression (sklearn.linear_model.BayesianRidge()), binary outcome can be encoded as +1 and -1. You will have to convert the continuous prediction back to binary (threshold it at zero) to calculate the test accuracy.

Task 3- Same as task 1, but PolynomialFeatures(degree=2) and LogisticRegression(), after applying k-means clustering on the training subjects (n_clusters=1-20, think of fixing the random_state in an appropriate way due to non-convexity). Plot the classification accuracy on the training and test sets (x axis=number of clusters, y axis=prediction accuracy; plot train/test as lines on the same plot using different colours).

Task 4- Same as task 1, but PolynomialFeatures(degree=2) and LogisticRegression(), after applying Gaussian mixture modelling clustering on the training subjects (n_clusters=1-20, think of fixing the random_state in an appropriate way due to non-convexity). Plot the

classification accuracy on the training and test sets (x axis=number of clusters, y axis=prediction accuracy; plot train/test as lines on the same plot using different colours).

Task 5- Like task 2, but plotting on y axis the number of miliseconds that are needed to do BayesianRidge().fit() for each degree (x axis).

Dataset B: use the nilearn Oasis dataset to predict a **continuous outcome (age)** based on structural brain scans

```
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker
from nilearn.image import index_img
from sklearn.preprocessing import StandardScaler
import nibabel as nib

brain_data = datasets.fetch_oasis_vbm(n_subjects=100)
crad = datasets.fetch_atlas_craddock_2012()
atlas_nii = index_img(crad['scorr_mean'], 42) # this needs to be modified for some tasks
print('%i regions found in this atlas' % (len(np.unique(atlas_nii.get_data().ravel())) - 1))

masker = NiftiLabelsMasker(labels_img= atlas_nii, standardize=True)
input_variables = masker.fit_transform(brain_data.gray_matter_maps)
output_variable = StandardScaler().fit_transform(brain_data.ext_vars.age[:, None])[:, 0] #
gives subject age on standard units after z-scoring
```

Note: use Ridge() and the R2 score (sklearn.metrics.r2_score) instead of LogisticRegression and accuracy. Use the lowest resolution atlas.

6- analogous to task 1 but predicting continuous subject age

7- analogous to task 2 but predicting continuous subject age. Now that the target is continuous you don't have to threshold the prediction.

8- analogous to task 3 but predicting continuous subject age

9- analogous to task 4 but predicting continuous subject age

10- analogous to task 5 but predicting continuous subject age